arXiv:2401.04081v2 [cs.LG] 26 Feb 2024

MoE-Mamba: Efficient Selective State Space Models with Mixture of Experts

Maciej Pióro [1 2] **Kamil Ciebiera** [1 3] **Krystian Król** [1 3] **Jan Ludziejewski** [1 3] **Michał Krutul** [1 3] **Jakub Krajewski** [1 3]
Szymon Antoniak **Piotr Miłoś** [1 4 3] **Marek Cygan** [3] **Sebastian Jaszczur** [1 3]

Abstract

State Space Models (SSMs) have become serious contenders in the field of sequential modeling, challenging the dominance of Transformers. At the same time, Mixture of Experts (MoE) has significantly improved Transformer-based Large Language Models, including recent state-of-the-art open models. We propose that to unlock the potential of SSMs for scaling, they should be combined with MoE. We showcase this on Mamba, a recent SSM-based model that achieves remarkable performance. Our model, MoE-Mamba, outperforms both Mamba and baseline Transformer-MoE. In particular, MoE-Mamba reaches the same performance as Mamba in $2.35\times$ *fewer training steps* while preserving the inference performance gains of Mamba against Transformer.

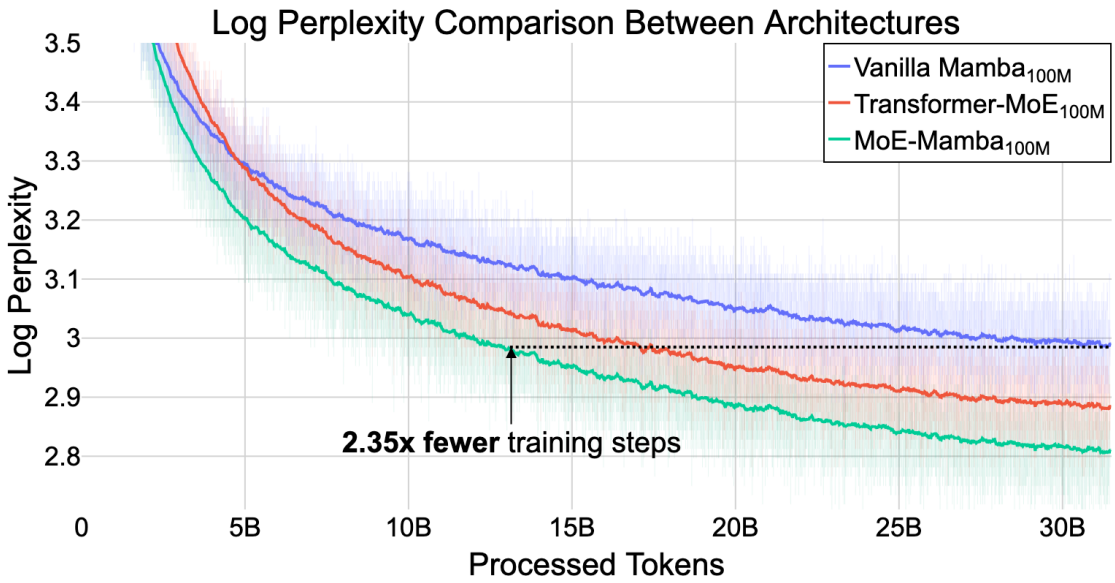

Figure 1. Log perplexity throughout the training. From top to bottom: $\text{Mamba}_{100\text{M}}$; $\text{Transformer-MoE}_{100\text{M}}$; $\text{MoE-Mamba}_{100\text{M}}$.

1. Introduction

Large Language Models (LLMs) have emerged as a cornerstone in the ongoing AI revolution (Brown et al., 2020; Chowdhery et al., 2023; Lewkowycz et al., 2022; OpenAI, 2023; Team, 2023). Their remarkable effectiveness is primarily attributed to the Transformer architecture (Vaswani et al., 2017) and training on an internet-wide scale, e.g., (TogetherComputer, 2023). Yet, questions remain: Should Transformers be the only architecture used for LLMs? Can we scale language models even further, and if so, how can this be achieved?

Detailed authors' contributions are listed in Appendix H. [1]IDEAS NCBR [2]Polish Academy of Sciences [3]University of Warsaw [4]Institute of Mathematics, Polish Academy of Sciences. Correspondence to: Sebastian Jaszczur <s.jaszczur@uw.edu.pl>.

Regarding the first question, State Space Models (SSMs), e.g., (Gu et al., 2022b; 2021; 2022a; Gupta et al., 2022; Li et al., 2022; Ma et al., 2022; Orvieto et al., 2023; Smith et al., 2023), have been increasingly gaining attention. This recognition is due to their capability for linear-time inference, highly parallelized training, and strong performance in tasks requiring long-context processing, such as those illustrated by the Long Range Arena (Tay et al., 2020). Notably, a recent addition to this category, Mamba (Gu & Dao, 2023), has shown impressive results through its selective mechanism and hardware-aware design, positioning it as a promising contender to the attention-based Transformer architecture.

Scaling is believed to be a critical factor in developing powerful AI systems (Sutton, 2019). The Mixture of Experts (MoE) approach (Jacobs et al., 1991), a set of techniques that enables an increase in model parameters with minimal impact on computational demands, plays a significant role. Due to their sparse activation, MoEs can be efficiently scaled up to trillions of parameters, as demonstrated by Shazeer et al. (2017); Fedus et al. (2022). MoE variants (Fedus et al., 2022; Du et al., 2022) are now routinely used in LLMs, as exemplified in the recent Mixtral model (Jiang et al., 2024).

In this paper, *we advocate that to unlock the potential of SSMs for scaling up, they should be combined with Mixture of Experts (MoE)*. To this end, we introduce **MoE-Mamba**, a model that combines Mamba (Gu & Dao, 2023) with a Switch layer (Fedus et al., 2022). MoE-Mamba enables efficiency gains of both SSMs and MoE, outperforming Mamba and Transformer-MoE, see Figure 1. Through comprehensive studies, we confirm that the effect is robust to the design choices and the number of experts. Our results indicate a very promising research direction that may allow scaling SSMs beyond tens of billions of parameters and compete with the largest SoTA language models.

In summary, our contributions are as follows:

- We introduce MoE-Mamba, a model that combines Mamba with a Mixture of Experts layer. MoE-Mamba enables efficiency gains of both SSMs and MoE while reaching the same performance as Mamba in $2.35\times$ fewer training steps.
- Via comprehensive studies, we confirm that the improvement achieved by MoE-Mamba is robust to varying model sizes, design choices, and the number of experts.
- We explore and compare multiple alternative methods of integrating Mixture of Experts within the Mamba block.

2. Related Work

State Space Models and Related Attention-Free Architectures State Space Models (SSMs) (Gu et al., 2022b; 2021; 2022a; Gupta et al., 2022; Li et al., 2022; Ma et al., 2022; Orvieto et al., 2023; Smith et al., 2023) form a family of architectures used for sequence modeling. Stemming from signal processing, these models can be seen as a combination of RNNs and CNNs (Gu & Dao, 2023). Although they potentially offer considerable benefits, a number of issues have been identified with SSMs (Gu et al., 2022b), preventing SSMs from becoming the leading architecture in the task of language modeling. However, recent breakthroughs (Gu et al., 2022b; Fu et al., 2023; Smith et al., 2023; Gu & Dao, 2023), have allowed deep SSMs to be increasingly competitive against Transformers (Vaswani et al., 2017). In particular, Mamba (Gu & Dao, 2023), studied in this paper, has shown impressive results through its selective mechanism and hardware-aware design, which allows scaling to billions of parameters while retaining computational efficiency and strong performance. Besides SSMs, numerous other architectures have been proposed that do not rely on the quadratic attention mechanism (Zhai et al., 2021; Poli et al., 2023; Sun et al., 2023; Peng et al., 2023).

Mixture of Experts Mixture of Experts (MoE) is a class of techniques that allow drastically increasing the number of parameters of a model without much impact on the FLOPs required for the model's training and inference. Introduced by Jacobs et al. (1991); Jordan & Jacobs (1993), MoE was applied in the context of NLP by Shazeer et al. (2017). MoE models benefit from sparse activation - for each token processed, only a subset of the model's parameters is used. Due to their computational demands, feed-forward layers in Transformers have become the standard target of various MoE techniques (Lepikhin et al., 2020; Fedus et al., 2022; Du et al., 2022; Zoph et al., 2022). Scaling properties and generalization abilities of MoE Transformers have been studied more closely by Artetxe et al. (2021); Clark et al. (2022); Krajewski et al. (2024).

More recently, MoE models have found their way onto the open scene (Xue et al., 2023; Jiang et al., 2024). In particular, the Mixtral 8×7B model (Jiang et al., 2024) fares comparably to Llama 2 70B (Touvron et al., 2023) while requiring only around $1/6$ of its inference computational budget.

Subsequently to the first version of our work, Anthony et al. (2024) presented an architecture similar to MoE-Mamba, showcasing the potential of connecting Mamba with MoE on downstream tasks, which validates our findings. In contrast to

their work, we run extensive ablations on the model architecture, number of experts, and other design choices. We also investigate the potential of integrating conditional computation into the Mamba block.

3. MoE-Mamba

In this section, we present the architecture details of our model, MoE-Mamba, see Figure 2. We start with a brief overview of the Mamba architecture, followed by a description of the MoE layer. Our main architecture is presented in Section 3.2, while sections 3.3 and 3.4 explore its variants and related approaches.

3.1. Preliminaries

Mamba Mamba (Gu & Dao, 2023) is a recently proposed SSM-based model that achieves remarkable, Transformer-like performance. By employing a work-efficient parallel scan, Mamba mitigates the impact of the sequential nature of recurrence, whereas fusing GPU operations removes the requirement to materialize the expanded state. Intermediate states necessary for backpropagation are not saved but recomputed during the backward pass, thus reducing memory requirements. The advantages of Mamba over the attention mechanism are especially prominent during inference, as not only is the computational complexity lowered, but also the memory usage is not dependent on the context length. Figure 3 shows the inner structure of a Mamba layer.

MoE Layer In our work, we follow the well-established (Zhao et al., 2023a; Sanseviero et al., 2023) and easy-to-implement Switch Transformer MoE design (Fedus et al., 2022) and leave consideration of other MoE designs for future work.

We assume N_{experts} experts $\{E_i\}_{i=1}^{N_{\text{experts}}}$, each being a trainable feed-forward network with the same number of parameters. For each token embedding x, we calculate scores $h(x) = Wx \in \mathbb{R}^{N_{\text{experts}}}$, where W is a trainable linear projection. These are normalized using softmax:

$$p_i(x) = \frac{\exp\left(h(x)_i\right)}{\sum_{i=1}^{N_{\text{experts}}} \exp\left(h(x)_i\right)}.$$

Prior to Switch, top-k routing selecting $k > 1$ most suitable experts for each token was deemed necessary. However, Switch successfully simplifies previous MoE approaches by setting $k = 1$. Namely, the output of the MoE layer for x is given by:

$$y = p_I E_I(x),$$

where $I = \operatorname{argmax}_i p_i(x)$.

During batched execution, e.g., in training, each batch contains N tokens. Following the standard procedure, in a case where the assignment of tokens to the experts is not perfect, i.e., some expert E_f is selected by more than N/N_{experts} tokens in the current batch, the excess tokens are dropped and not updated (capacity factor $= 1$). To further encourage an even distribution of tokens to experts, load balancing loss as described by Fedus et al. (2022) with weight $\alpha = 0.01$ is added to the training objective.

3.2. MoE-Mamba Architecture

The vanilla Mamba architecture consists of multiple Mamba blocks stacked one after another, with each layer's output being added to the residual stream; see Figure 2. In MoE-Mamba, we interleave Mamba layers with MoE layers (see Figure 2). Note that the vanilla Mamba does not use feed-forward layers.

In this way, MoE-Mamba separates unconditional processing of every token by the Mamba layer - which can efficiently integrate the whole sequence context into an internal representation - and conditional processing by an MoE layer that can apply the most relevant expert (and thus the subset of parameters) for each token. The idea of interleaving conditional and unconditional processing is used in some MoE-based models, typically by alternating vanilla and MoE feed-forward layers (Lepikhin et al., 2020; Fedus et al., 2022).

3.3. Parallel MoE-Mamba

Apart from interleaving MoE layers with Mamba layers, we explore another design, inspired by Wang (2021) and Chowdhery et al. (2023) in which MoE layer is executed in parallel with Mamba (see Figure 3). It achieves positive results, albeit worse than MoE-Mamba.

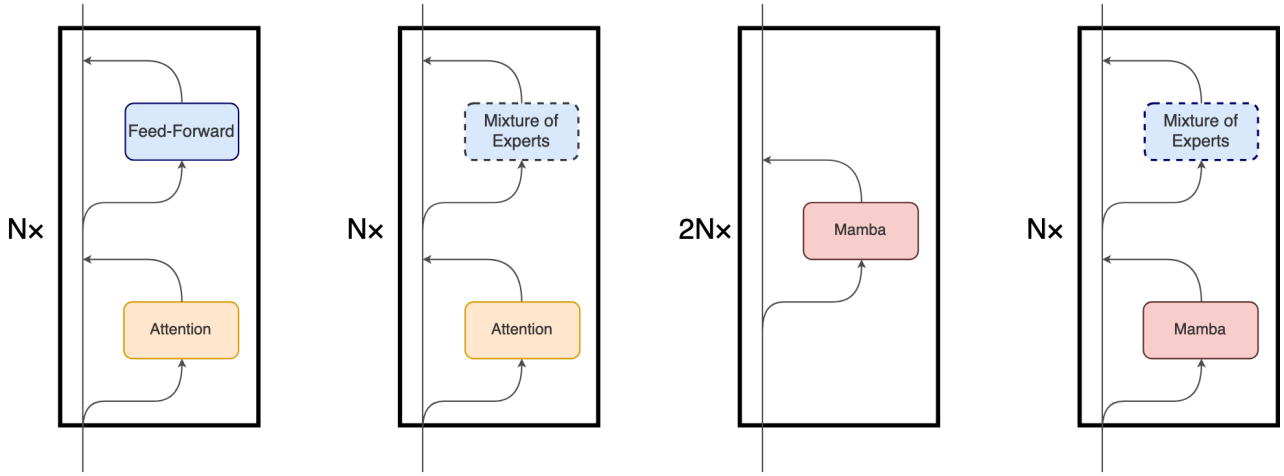

Figure 2. Diagrams of the architectures. From the left: vanilla Transformer, Transformer-MoE, Mamba, MoE-Mamba.

Table 1. Comparison between different architectures. The $\square_{25\text{M}}$ models were trained on ca. 10B tokens and the $\square_{100\text{M}}$ models were trained on ca. 30B tokens. Note that the parameter counts exclude embedding and output (unembedding) layers (for further discussion of reporting either non-embedding or all parameters, see Appendix E). The numbers of total and active parameters are not matched exactly between similarly sized models due to, among other reasons, the MoE models including routers and Mamba layer not containing precisely $6d^2_{\text{model}}$ parameters - a design choice we did not want to modify. We consider those differences to be too small to be significant for our results.

Model	# Parameters	# Active Parameters per Token	Final Log Perplexity	Speedup Over Vanilla Mamba (Training Steps)
$\text{Mamba}_{25\text{M}}$	27M	27M	3.34	1
$\text{MoE-Mamba}_{25\text{M}}$ (ours)	542M	26M	**3.19**	**1.76**
$\text{Transformer-MoE}_{25\text{M}}$	545M	25M	3.23	1.56
$\text{Transformer}_{25\text{M}}$	25M	25M	3.43	>1
$\text{Mamba}_{100\text{M}}$	121M	121M	2.99	1
$\text{MoE-Mamba}_{100\text{M}}$ (ours)	2439M	117M	**2.81**	**2.35**
$\text{Transformer-MoE}_{100\text{M}}$	2454M	114M	2.88	1.79

3.4. Modifying Mamba Block

In addition to attaching a separate MoE layer to Mamba, we also conducted other experiments, modifying the original block design by Gu & Dao (2023) to feature conditional MoE computation. Some of the designs show improvements over the baseline architecture and suggest promising future research directions.

4. Experiments

In this section we provide empirical validation of our hypothesis that interleaving Mamba with MoE can improve the performance of Mamba. Our main result, see Figure 1, shows that MoE-Mamba needs $2.35\times$ *fewer training steps* to reach the same performance as Mamba. We also provide a detailed analysis of our design choices.

4.1. Training Setup

We compare MoE-Mamba to three baselines: Mamba, Transformer, and Transformer-MoE. All models in our experiments are decoder-only.

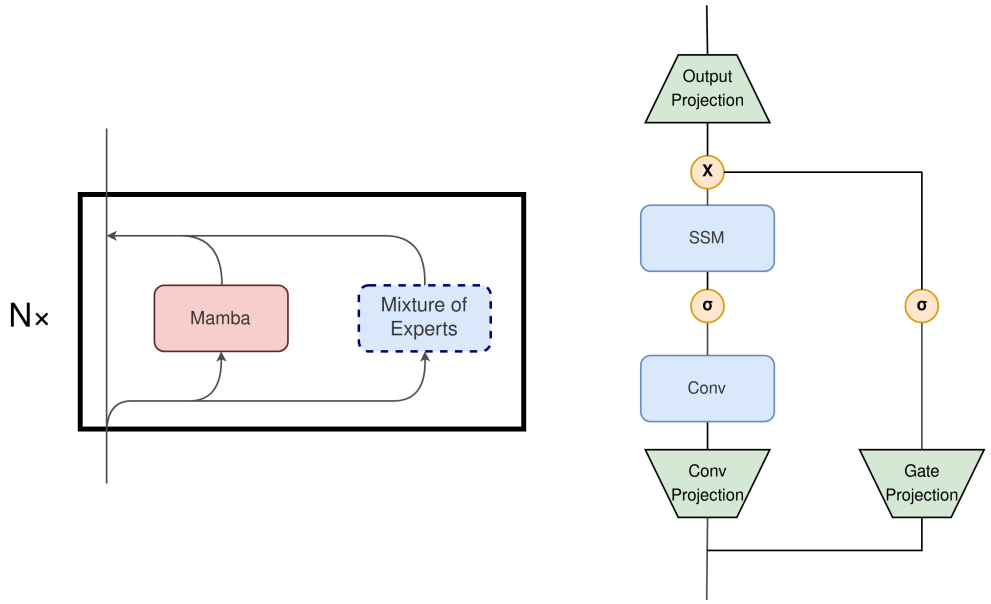

Figure 3. Diagram of Parallel MoE-Mamba architecture (left) and Mamba Block (right). The outputs of the Gate and Conv Projections are E (expansion factor) times bigger than the input, i.e., Conv and SSM operate on vectors $\in \mathbb{R}^{E \cdot d_{\text{model}}}$. Vanilla Mamba assumes $E = 2$ (Gu & Dao, 2023). Expansion factor E determines how much the input vector is scaled up by Gate and Conv Projection and then scaled down by Output Projection, and because of that, it is also proportional to the number of FLOPs and parameters in the Mamba layer.

In the standard Transformer architecture, a single attention layer contains $4d^2_{\text{model}}$ parameters, whereas a feed-forward layer contains $8d^2_{\text{model}}$ parameters. A single Mamba layer contains slightly over $6d^2_{\text{model}}$ (Gu & Dao, 2023) parameters. To be able to compare MoE-Mamba to Transformer-based and Mamba baselines, we scale down the size of each expert in our model (we set $d_{\text{expert}} = 3d_{\text{model}}$). This way, we can keep both the number of blocks and the number of active parameters per token roughly the same in all models of similar size. Active parameters denote those used to calculate the output for a given token (e.g., typically, only one expert in each MoE layer is active). For a discussion of the relation of active parameters and FLOPs, see Appendix B.

Due to computational constraints, we perform most of our experiments on smaller, $\square_{\text{25M}}$ models and validate our findings on $\square_{\text{100M}}$ models.

We train the models on C4 dataset (Raffel et al., 2020) on the next token prediction task using cross entropy as the loss function. We use EMA-smoothed ($\alpha = 0.001$) training log perplexity as the comparison metric for both final loss and speedup measurements as it is a more fine-grained comparison metric than test log perplexity. The test log perplexity comparison for $\square_{\text{100M}}$ models can be found in Appendix G. All models use the GPT2 tokenizer (Radford et al., 2019). We tune the learning rate separately for all $\square_{\text{25M}}$ models and reuse it when training their $\square_{\text{100M}}$ counterparts. When training Transformer-MoE$_{\text{100M}}$, we divide the learning rate by two due to repeated instabilities. See Appendix A for further details and hyperparameters. The main experiments, described in section 4.2, use around 10B tokens for $\square_{\text{25M}}$ models and around 30B tokens for $\square_{\text{100M}}$ models. The experiments described in further sections use 1B tokens.

4.2. Main Results

Table 1 presents the comparison between training results of MoE-Mamba and baselines; see also Figure 1 for log perplexity curves. MoE-Mamba shows a remarkable improvement over the vanilla Mamba model across both model sizes. Notably, MoE-Mamba$_{\text{100M}}$ was able to perform on par with vanilla Mamba$_{\text{100M}}$ with $2.35\times$ speedup in terms of processed tokens. For $\square_{\text{25M}}$ model size, those performance gains are lower, probably due to a lower number of training tokens. More generally, we observe that the gains increase over the training, oscillating around $1.6\times - 1.9\times$ for $\square_{\text{25M}}$ models after the initial training period. Further discussion of the speedup can be found in Appendix D. We observe that MoE-Mamba performs better than the corresponding Transformer-MoE, which strengthens the findings by Gu & Dao (2023) that Mamba is a competitive alternative to the Transformer.

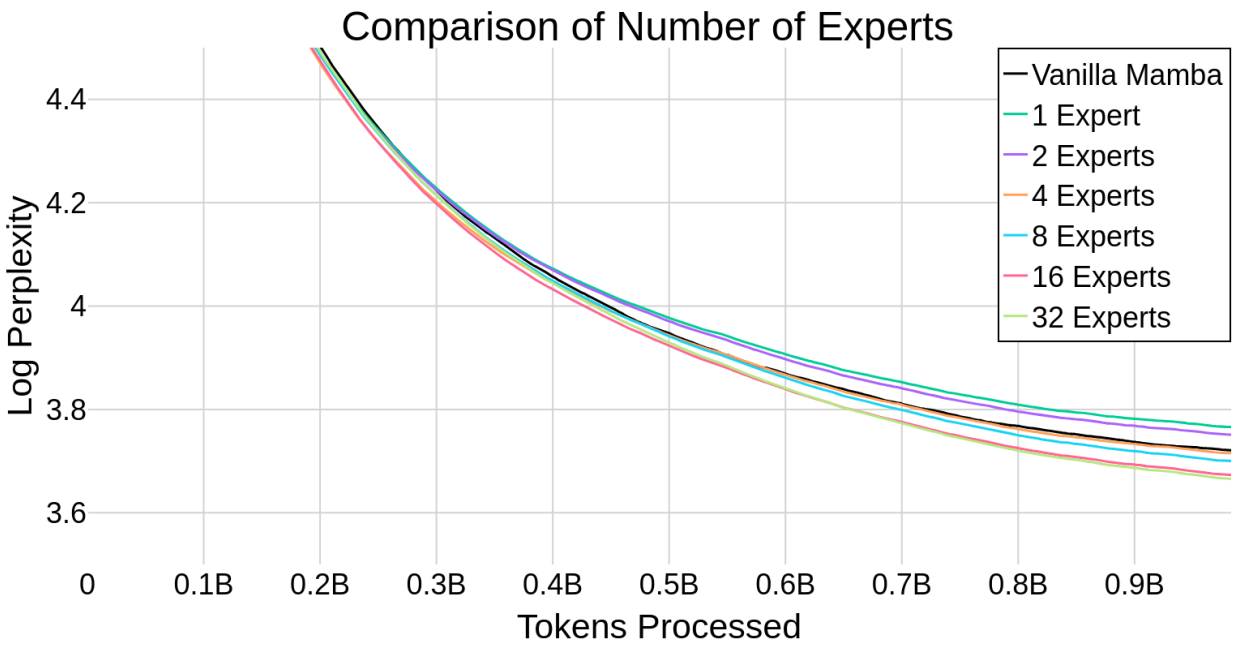

Figure 4. Smoothed training loss (log perplexity) for a differing number of experts for MoE-Mamba with ca. 26M active non-embedding parameters. The final log perplexity improves monotonically as the number of experts increases.

Table 2. Comparison of different ratios of parameters between Mamba and MoE. The $E = 2$ corresponds to MoE-Mamba$_{25\text{M}}$. The total number of parameters in all models is 542M and the number of active parameters per token is 26M.

Ratio $N_{\text{Mamba}}^{\text{act. params}} : N_{\text{MoE}}^{\text{act. params}}$	Expansion Factor E (Mamba)	Expert Size	Number of Experts
$1:5$	$\frac{2}{3}$	2560	19
$2:4$	$1\frac{2}{3}$	2048	24
$3:3$	2	1536	32
$4:2$	$2\frac{2}{3}$	1024	48
$5:1$	$3\frac{1}{3}$	512	96

4.3. Optimal Ratio of Active Parameters in Mamba and MoE

In this section, we investigate the optimal ratio of active parameters in the Mamba layer to active parameters in the MoE layer while keeping the total number of parameters fixed. Under these constraints, a given ratio determines the so-called expansion factor E of the Mamba layer, the number of experts, and their size as detailed in Table 2 (see also Figure 3 for Mamba design).

The results are presented in Figure 5. We observe that increasing the number of active Mamba parameters improves the performance. However, the gains become marginal after reaching the $3:3$ ratio, and higher ratios are impractical due to inefficient hardware utilization and high routing costs caused by a large number of experts. We default to this choice in all other experiments.

4.4. Alternative Designs

Parallel MoE-Mamba Inspired by Wang (2021) and Chowdhery et al. (2023), we experiment with an alternative block design in which the MoE feed-forward layer and the Mamba layer are placed in parallel instead of sequentially (see Figure 3). We compare this design to MoE-Mamba for various numbers of experts; see Figure 6. MoE-Mamba outperforms this

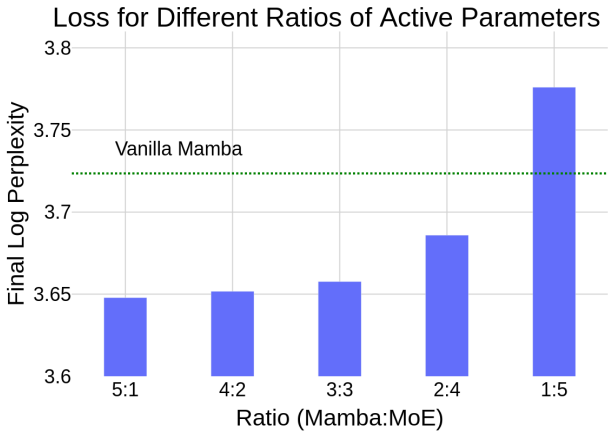

Figure 5. Final log perplexity at different ratios of active Mamba-to-MoE active parameters. Note that MoE contains the majority of the total parameters in each model. For further discussion of the ratios explored, see Appendix F.

Table 3. Comparison of different variants of MoE in Mamba - final log perplexity (1B tokens).

Model Name / Modified Projection	**MoE in Mamba**	
	All Layers	Every Other Layer
Vanilla Mamba	3.72	
MoE-Mamba (16 experts)	**3.67**	
Conv Projection	3.79	3.71
Gate Projection	3.89	3.70
Output Projection	4.05	3.70
Conv + Gate Projection	3.95	3.72
Conv + Output Projection	4.17	3.76
Gate + Output Projection	4.16	3.88
Conv + Gate + Output Projection	4.39	3.88

variant in all tested settings. The parallel MoE-Mamba matches vanilla Mamba when $N_{\text{experts}} \geq 8$ while requiring between 2 and 4 times as many experts and total parameters to match the performance of the sequential variant. It may be an attractive alternative at larger scales due to potentially enabling more efficient use of hardware due to different communication (Wang, 2021) or fused input matrix multiplications (Chowdhery et al., 2023).

Inner MoE Pursuing a uniform layer design, we experimented with replacing each of the three linear projections within the Mamba block with an MoE layer; see Figure 3. Enumerating all the possible placements results in $2^3 - 1 = 7$ possible designs (we discard one combination that would feature no MoE inside the block). We maintain a similar number of total parameters and FLOPs in all models by assuring the total number of expert feed-forward layers in a block sums up to 24 regardless of the placement, i.e., the 24 experts are split evenly between one, two or three MoE's inside the block. Inspired by Fedus et al. (2022), we also performed experiments in which only half of the Mamba blocks were modified to include MoE, but the number of experts was increased to 48 to maintain the total number of parameters.

Three of the designs (Table 3) achieved results marginally better than vanilla Mamba, with none outperforming MoE-Mamba. These results suggest the most promising research directions in future work.

4.5. Number of Experts

Figure 4 shows the training runs for different numbers of experts. The results show that our approach scales favorably with the number of experts. MoE-Mamba outperforms vanilla Mamba, when $N_{\text{experts}} \geq 4$. We obtain the best result with 32 experts and expect further gains with even more experts.

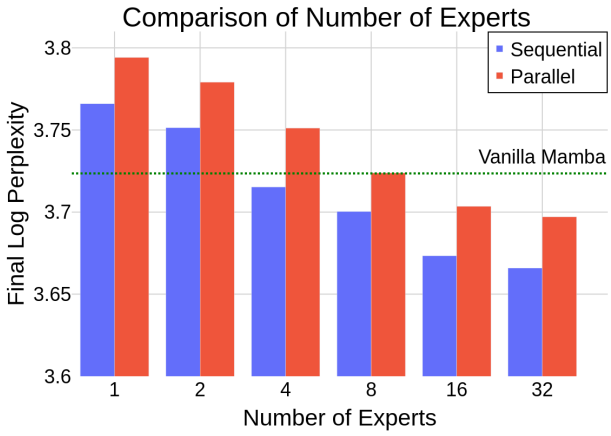

Figure 6. Final log perplexity comparison for varying number of experts in sequential and parallel MoE-Mamba

Table 4. Log perplexity after 1B tokens for various numbers of experts. Note that the parameter counts exclude the embedding and output (unembedding) layers.

Number of Experts	# Parameters	# Active Parameters per Token	Log Perplexity After 1B Tokens	Speedup Over Vanilla Mamba (Training Steps)
N/A - Vanilla Mamba	27M	27M	3.72	1
1	26M	26M	3.75	<1
4 experts	64M	26M	3.72	1.03
8 experts	114M	26M	3.70	1.10
16 experts	215M	26M	3.67	1.21
32 experts	416M	26M	**3.67**	**1.23**

Interestingly, models with a small number of experts perform worse than vanilla Mamba. This is consistent with Gu & Dao (2023) reporting that Mamba interleaved with feed-forward layers (which corresponds to a single-expert MoE layer) is worse than vanilla Mamba.

4.6. Accuracy and Perplexity

We observed that throughout the training of a variant of one of our smaller models, MoE-Mamba$_{25M}$ with 32 instead of 42 experts as presented in section 4.2, it maintains a lower perplexity than our strongest baseline (Transformer-MoE). However, at the same time, Transformer-MoE consistently achieves higher accuracy than MoE-Mamba. We conjecture that this might be due to the fact that attention-based models are able to copy tokens verbatim, unlike SSM-based models, whose similar abilities might be hindered by the compression of the history into a finite hidden state. We present accuracy and loss (log perplexity) plots alongside further discussion of those results in Appendix C.

5. Future Work and Limitations

Scaling In this work, we perform experiments on models with the number of active parameters per token smaller than 1B, with total parameters up to 2.4B. Since MoE has enabled Transformers to be scaled to unprecedented sizes (Fedus et al., 2022), we will be excited to see the impact of scaling on the approaches proposed in our work. Developing scaling laws would be instrumental in this endeavor.

Integrating MoE Into the Mamba Layer Our experiments show that interleaving the Mamba layer with a performant sparse MoE feed-forward layer results in a promising model. However, in the dense setting, Mamba performs slightly better without the feed-forward layer. This suggests that integrating sparse computation within the Mamba layer itself could yield even better results while conserving a simple, homogeneous architecture. Our experiments, detailed in section 4.4, warrant

some optimism, and we expect this line of research to remain relevant.

Exploration of Different Types of MoE in MoE-Mamba While we base our design on the commonly used Switch (Fedus et al., 2022), numerous other MoE architectures have been proposed. Not only may those designs perform better overall, but it is possible that a different type of MoE will be optimal when combined with SSMs. Among possible changes in this regard there are Expert-Choice routers (Zhou et al., 2022), fully differentiable architectures (Puigcerver et al., 2023; Antoniak et al., 2023), varying number of experts and their granularity, (Clark et al., 2022; Krajewski et al., 2024), and other modifications.

Distillation Some works, e.g., (Fedus et al., 2022), have shown that MoE layers can be distilled back to feed-forward layers. We expect similar results for MoE-Mamba. Interestingly, the findings by Gu & Dao (2023) indicate that a Mamba module can emulate feed-forward layers well. This raises the question of whether MoE can be distilled into a vanilla Mamba module and how that would be achieved.

Synergies We leave for future work more in-depth studies of synergies of Mamba and MoE. We suspect that there might be efficiency gains growing with the context length due to better hardware utilization; as for inference, Mamba alleviates computation and memory throughput issues stemming from larger context sizes, while MoE alleviates those same issues stemming from increasing number of parameters and knowledge stored in the model. This synergy may allow for unprecedented scaling of language models both in the number of parameters and length of the input/output.

Mamba and Attention Mechanism Mamba and Transformers make different trade-offs during data processing. This results in a different set of strengths and weaknesses, e.g., Mamba can process very long inputs but might struggle with tasks requiring detailed knowledge of the past input (e.g., some instances of copying). It would be interesting to explore combining those two architectures to achieve the best of both worlds.

Long Context Utilization Mamba and other SSMs are praised for their ability to process long context. However, the extent to which they can utilize it effectively and techniques for improving the utilization have not yet been studied in depth. To that end, some methods developed for Transformers (Shi et al., 2023; Tworkowski et al., 2023; Staniszewski et al., 2024) might be applicable.

Other Modalities This work explores one direction in which Mamba can be extended. Mamba is a general architecture, and it is not limited to language modeling. We expect that it will be possible to apply MoE-Mamba to other tasks, like non-textual sequence modeling presented by Gu & Dao (2023), and different modalities, such as vision, with initial work presented by Zhu et al. (2024).

6. Conclusions

In this work, we presented the first integration of Mixture of Experts with Mamba architecture, MoE-Mamba. This novel method shares the inference benefits of Mamba while requiring $2.35\times$ fewer training steps to reach the same performance. We showed possible ways of combining those techniques and positively verified performance improvements achieved with their combination. We confirmed with experiments on models up to 2.4B parameters and training lengths up to 30B tokens that those improvements over Mamba are robust to model sizes, length of training, and the number of experts.

In addition to the above, we explored and evaluated numerous alternative designs integrating Mixture of Experts within the Mamba block. While none of those variants outperformed MoE-Mamba, we think that those investigations can help prune ineffective research directions and point to promising ones.

Our work opens a new research direction of combining Mixture of Experts with State Space Models. We believe that this path will enable more efficient scaling to even larger language models.

Acknowledgments

We would like to express sincere gratitude to Tomasz Odrzygóźdź for the engineering contributions made to our shared repository. We also thank Piotr Sankowski for creating a supportive environment and providing research direction.

This work was funded by IDEAS NCBR, which also provided significant computational resources and a supportive research environment. The research was supported by PL-Grid infrastructure (grant PLG/2023/016148). We acknowledge snakes and experts as essential to our work. We also benefited from the Entropy cluster (hosted at the Faculty of Mathematics, Informatics and Mechanics of the University of Warsaw) funded by NVIDIA, Intel, the Polish National Science Center grant 2022/45/N/ST6/02222, and ERC Starting Grant TOTAL. Marek Cygan was partially supported by an NCBiR grant POIR.01.01.01-00-0392/17-00.

References

Anthony, Q., Tokpanov, Y., Glorioso, P., and Millidge, B. Blackmamba: Mixture of experts for state-space models, 2024.

Antoniak, S., Jaszczur, S., Krutul, M., Pióro, M., Krajewski, J., Ludziejewski, J., Odrzygóźdź, T., and Cygan, M. Mixture of tokens: Efficient llms through cross-example aggregation, 2023.

Artetxe, M., Bhosale, S., Goyal, N., Mihaylov, T., Ott, M., Shleifer, S., Lin, X. V., Du, J., Iyer, S., Pasunuru, R., et al. Efficient large scale language modeling with mixtures of experts. *arXiv preprint arXiv:2112.10684*, 2021.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. *CoRR*, abs/2005.14165, 2020. URL https://arxiv.org/abs/2005.14165.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

Clark, A., de las Casas, D., Guy, A., Mensch, A., Paganini, M., Hoffmann, J., Damoc, B., Hechtman, B., Cai, T., Borgeaud, S., van den Driessche, G., Rutherford, E., Hennigan, T., Johnson, M., Millican, K., Cassirer, A., Jones, C., Buchatskaya, E., Budden, D., Sifre, L., Osindero, S., Vinyals, O., Rae, J., Elsen, E., Kavukcuoglu, K., and Simonyan, K. Unified scaling laws for routed language models, 2022.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

Du, N., Huang, Y., Dai, A. M., Tong, S., Lepikhin, D., Xu, Y., Krikun, M., Zhou, Y., Yu, A. W., Firat, O., et al. Glam: Efficient scaling of language models with mixture-of-experts. In *International Conference on Machine Learning*, pp. 5547–5569. PMLR, 2022.

Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., DasSarma, N., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. https://transformer-circuits.pub/2021/framework/index.html.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022.

Fu, D. Y., Dao, T., Saab, K. K., Thomas, A. W., Rudra, A., and Ré, C. Hungry hungry hippos: Towards language modeling with state space models, 2023.

Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces, 2023.

Gu, A., Johnson, I., Goel, K., Saab, K., Dao, T., Rudra, A., and Ré, C. Combining recurrent, convolutional, and continuous-time models with linear state-space layers, 2021.

Gu, A., Goel, K., Gupta, A., and Ré, C. On the parameterization and initialization of diagonal state space models. *Advances in Neural Information Processing Systems*, 35:35971–35983, 2022a.

Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces, 2022b.

Gupta, A., Gu, A., and Berant, J. Diagonal state spaces are as effective as structured state spaces. *Advances in Neural Information Processing Systems*, 35:22982–22994, 2022.

Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive mixtures of local experts. *Neural Computation*, 3(1): 79–87, 1991. doi: 10.1162/neco.1991.3.1.79.

Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Hanna, E. B., Bressand, F., Lengyel, G., Bour, G., Lample, G., Lavaud, L. R., Saulnier, L., Lachaux, M.-A., Stock, P., Subramanian, S., Yang, S., Antoniak, S., Scao, T. L., Gervet, T., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. Mixtral of experts, 2024.

Jordan, M. and Jacobs, R. Hierarchical mixtures of experts and the em algorithm. In *Proceedings of 1993 International Conference on Neural Networks (IJCNN-93-Nagoya, Japan)*, volume 2, pp. 1339–1344 vol.2, 1993. doi: 10.1109/IJCNN.1993.716791.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models, 2020.

Krajewski, J., Ludziejewski, J., Adamczewski, K., Pióro, M., Krutul, M., Antoniak, S., Ciebiera, K., Król, K., Odrzygóźdź, T., Sankowski, P., Cygan, M., and Jaszczur, S. Scaling laws for fine-grained mixture of experts, 2024.

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., and Soricut, R. Albert: A lite bert for self-supervised learning of language representations, 2020.

Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. Gshard: Scaling giant models with conditional computation and automatic sharding, 2020.

Lewkowycz, A., Andreassen, A. J., Dohan, D., Dyer, E., Michalewski, H., Ramasesh, V. V., Slone, A., Anil, C., Schlag, I., Gutman-Solo, T., Wu, Y., Neyshabur, B., Gur-Ari, G., and Misra, V. Solving quantitative reasoning problems with language models. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=IFXTZERXdM7.

Li, Y., Cai, T., Zhang, Y., Chen, D., and Dey, D. What makes convolutional models great on long sequence modeling? *arXiv preprint arXiv:2210.09298*, 2022.

Loshchilov, I. and Hutter, F. Decoupled weight decay regularization, 2019.

Ma, X., Zhou, C., Kong, X., He, J., Gui, L., Neubig, G., May, J., and Zettlemoyer, L. Mega: moving average equipped gated attention. *arXiv preprint arXiv:2209.10655*, 2022.

Olsson, C., Elhage, N., Nanda, N., Joseph, N., DasSarma, N., Henighan, T., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Johnston, S., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html.

OpenAI. Gpt-4 technical report, 2023.

Orvieto, A., Smith, S. L., Gu, A., Fernando, A., Gulcehre, C., Pascanu, R., and De, S. Resurrecting recurrent neural networks for long sequences. *arXiv preprint arXiv:2303.06349*, 2023.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library, 2019.

Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., Cao, H., Cheng, X., Chung, M., Grella, M., GV, K. K., He, X., Hou, H., Lin, J., Kazienko, P., Kocon, J., Kong, J., Koptyra, B., Lau, H., Mantri, K. S. I., Mom, F., Saito, A., Song, G., Tang, X., Wang, B., Wind, J. S., Wozniak, S., Zhang, R., Zhang, Z., Zhao, Q., Zhou, P., Zhou, Q., Zhu, J., and Zhu, R.-J. Rwkv: Reinventing rnns for the transformer era, 2023.

Poli, M., Massaroli, S., Nguyen, E., Fu, D. Y., Dao, T., Baccus, S., Bengio, Y., Ermon, S., and Ré, C. Hyena hierarchy: Towards larger convolutional language models, 2023.

Puigcerver, J., Riquelme, C., Mustafa, B., and Houlsby, N. From sparse to soft mixtures of experts, 2023.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551, 2020.

Sanseviero, O., Tunstall, L., Schmid, P., Mangrulkar, S., Belkada, Y., and Cuenca, P. Mixture of experts explained, 2023. URL https://huggingface.co/blog/moe.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.

Shi, W., Min, S., Lomeli, M., Zhou, C., Li, M., James, R., Lin, X. V., Smith, N. A., Zettlemoyer, L., Yih, S., and Lewis, M. In-context pretraining: Language modeling beyond document boundaries, 2023.

Smith, J. T. H., Warrington, A., and Linderman, S. W. Simplified state space layers for sequence modeling, 2023.

Staniszewski, K., Tworkowski, S., Jaszczur, S., Michalewski, H., Łukasz Kuciński, and Miłoś, P. Structured packing in llm training improves long context utilization, 2024.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding, 2023.

Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. Retentive network: A successor to transformer for large language models, 2023.

Sutton, R. The bitter lesson. *Incomplete Ideas (blog)*, 13(1), 2019.

Tay, Y., Dehghani, M., Abnar, S., Shen, Y., Bahri, D., Pham, P., Rao, J., Yang, L., Ruder, S., and Metzler, D. Long range arena: A benchmark for efficient transformers. *arXiv preprint arXiv:2011.04006*, 2020.

Team, G. Gemini: A family of highly capable multimodal models, 2023.

TogetherComputer. Redpajama: An open source recipe to reproduce llama training dataset, April 2023. URL https://github.com/togethercomputer/RedPajama-Data.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models, 2023.

Turc, I., Chang, M.-W., Lee, K., and Toutanova, K. Well-read students learn better: On the importance of pre-training compact models, 2019.

Tworkowski, S., Staniszewski, K., Pacek, M., Wu, Y., Michalewski, H., and Miłoś, P. Focused transformer: Contrastive training for context scaling, 2023.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL http://arxiv.org/abs/1706.03762.

Wang, B. Mesh-Transformer-JAX: Model-Parallel Implementation of Transformer Language Model with JAX. https://github.com/kingoflolz/mesh-transformer-jax, May 2021.

Xue, F., Zheng, Z., Fu, Y., Ni, J., Zheng, Z., Zhou, W., and You, Y. Openmoe: Open mixture-of-experts language models. https://github.com/XueFuzhao/OpenMoE, 2023.

Zhai, S., Talbott, W., Srivastava, N., Huang, C., Goh, H., Zhang, R., and Susskind, J. An attention free transformer, 2021.

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J.-Y., and Wen, J.-R. A survey of large language models, 2023a.

Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.-C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., Desmaison, A., Balioglu, C., Damania, P., Nguyen, B., Chauhan, G., Hao, Y., Mathews, A., and Li, S. Pytorch fsdp: Experiences on scaling fully sharded data parallel, 2023b.

Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., Zhao, V., Dai, A., Chen, Z., Le, Q., and Laudon, J. Mixture-of-experts with expert choice routing, 2022.

Zhu, L., Liao, B., Zhang, Q., Wang, X., Liu, W., and Wang, X. Vision mamba: Efficient visual representation learning with bidirectional state space model, 2024.

Zoph, B., Bello, I., Kumar, S., Du, N., Huang, Y., Dean, J., Shazeer, N., and Fedus, W. St-moe: Designing stable and transferable sparse expert models, 2022.

A. Hyperparameters and Training Setup

Basic model hyperameters (d_{model}, d_{ff}, the number of attention heads, the number of layers) used in this work were inspired by BERT (Devlin et al., 2019; Turc et al., 2019), with the $\square_{25\text{M}}$ models being equivalent to $\text{BERT}_{\text{MEDIUM}}$ and $\square_{100\text{M}}$ models copying $\text{BERT}_{\text{BASE}}$ configuration while increasing the number of blocks from 12 to 16. The learning rate schedule, as well as weight decay and gradient clipping values were set per community's standard practices. We used the AdamW optimizer (Loshchilov & Hutter, 2019). We tune the maximum learning rate value for each of the $\square_{25\text{M}}$ models separately and divide it by 2 when training $\square_{100\text{M}}$ counterparts. We train the models using PyTorch (Paszke et al., 2019) and utilize FSDP (Zhao et al., 2023b) for facilitating multi-GPU setup.

Table 5. Hyperparameters ($\square_{25\text{M}}$ Models). In Transformer models we use Rotary Position Embedding (Su et al., 2023).

Hyperparameter		Transformer$_{25\text{M}}$	Mamba$_{25\text{M}}$	Transformer-MoE$_{25\text{M}}$	MoE-Mamba$_{25\text{M}}$
Model	Total Blocks	8	16	8	8
	d_{model}	512	512	512	512
	# Parameters	25M	27M	545M	542M
	# Active Parameters per Token	25M	27M	25M	26M
Feed-Forward	d_{ff}	2048	-	-	-
Mixture of Experts	d_{expert}	-	-	2048	1536
	N_{experts}	-	-	32	42
Position Embedding		RoPE	-	RoPE	-
Attention	N_{heads}	8	-	8	-
Training	Training Steps	150K	150K	150K	150K
	Context Length	1024	1024	1024	1024
	Batch Size	64	64	64	64
	Max Learning Rate	5e-4	1e-3	5e-4	5e-4
	LR Warmup	1%	1%	1%	1%
	LR Schedule	Cosine	Cosine	Cosine	Cosine
	Final LR Ratio	0.1	0.1	0.1	0.1
	Weight Decay	0.1	0.1	0.1	0.1
	Gradient Clipping	0.5	0.5	0.5	0.5

B. Active Parameters vs FLOPs

In this work, we report the number of active parameters (excluding embedding and unembedding layers) and not the number of floating-point operations (FLOPs), following (Zhou et al., 2022). Both numbers will be roughly proportional (Kaplan et al., 2020), but the number of FLOPs is both harder to calculate and less relevant for hardware-aware architecture like Mamba with its optimizations, especially during inference.

Table 6. Hyperparameters ($\square_{100\text{M}}$ Models). In Transformer-MoE$_{100\text{M}}$ we use Rotary Position Embedding (Su et al., 2023).

Hyperparameter		Mamba$_{100\text{M}}$	Transformer-MoE$_{100\text{M}}$	MoE-Mamba$_{100\text{M}}$
Model	Total Blocks	32	16	16
	d_{model}	768	768	768
	# Parameters	121M	2454M	2439M
	# Active Parameters per Token	121M	114M	117M
Mixture of Experts	d_{expert}	-	3072	2304
	N_{experts}	-	32	42
Position Embedding		-	RoPE	-
Attention	N_{heads}	-	12	-
Training	Training Steps	30K	30K	30K
	Context Length	1024	1024	1024
	Batch Size	1024	1024	1024
	Max Learning Rate	1e-3	2.5e-4	5e-4
	LR Warmup	1%	1%	1%
	LR Schedule	Cosine	Cosine	Cosine
	Final LR Ratio	0.1	0.1	0.1
	Weight Decay	0.1	0.1	0.1
	Gradient Clipping	0.5	0.5	0.5

Table 7. Comparison of sequential and parallel MoE-Mamba - final log perplexity (1B tokens).

# of Experts	MoE-Mamba	
	Sequential	Parallel
1	3.76	3.79
2	3.74	3.77
4	3.71	3.74
8	3.69	3.72
16	3.67	3.70
32	**3.66**	3.69

C. Accuracy and Perplexity

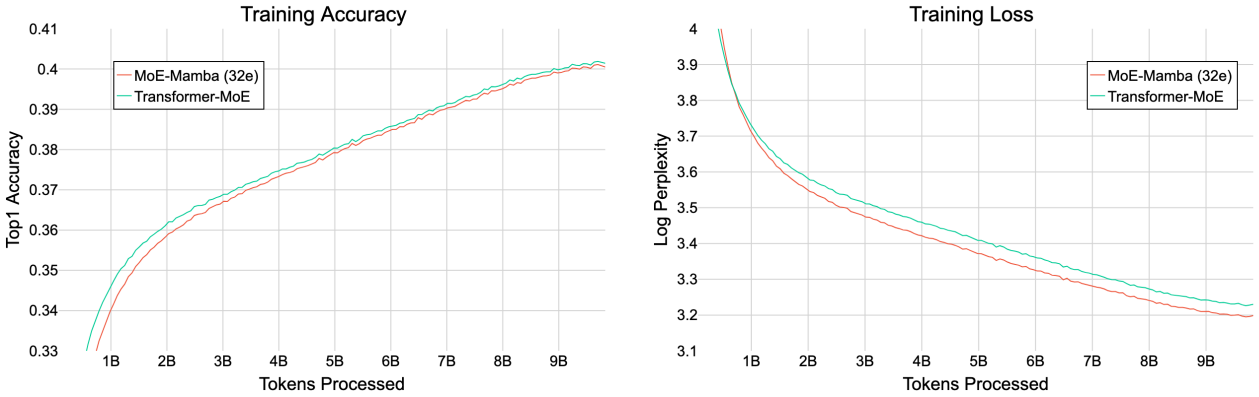

Figure 7. Discrepancy between accuracy and log perplexity: MoE-Mamba$_{25\mathrm{M}}$ with 32 experts and Transformer$_{25\mathrm{M}}$. Note that MoE-Mamba with 32 experts has fewer total parameters than the Transformer.

As mentioned in section 4.6, we have observed a curious case of metric inconsistency between two models that achieved similar performance but were based on different architectures. We hypothesize that this discrepancy hints at a potential failure mode of Mamba and other SSMs. Due to the compression of the history into a finite hidden state, their ability for verbatim token-copying is limited. The related ability to predict the token $[B]$ given a prefix $...[A][B]...[A]$ (where $[A]$, $[B]$ can be any tokens) has been mechanistically studied by Elhage et al. (2021) and has been conjectured to be responsible for Transformer's remarkable in-context learning capabilities (Olsson et al., 2022).

Peng et al. (2023) mention that their attention-free model, RWKV, may have limited performance on tasks that require recalling precise information over long contexts due to a fixed-sized hidden state, a property that Mamba and other SSMs share. However, since the perplexity of Mamba can match the perplexity of a similarly-sized Transformer, we can suspect that Mamba compensates for that failure mode in other ways and might show a relative advantage on other tasks when compared to Transformer. In particular, it might outperform Transformers in 0-shot tasks in contrast to tasks allowing few-shot demonstrations or requiring in-context learning.

D. Relation between Speedup and Training Time

In our experiments, we notice that generally, as the training continues, the speedup of MoE-Mamba compared to vanilla Mamba increases (see Fig. 8). That is, the ratio

$$\text{speedup}(l) = \frac{\text{\# processed tokens vanilla Mamba took to reach loss } l}{\text{\# processed tokens MoE-Mamba took to reach loss } l}$$

increases as l decreases. Speedup in $\Box_{25\mathrm{M}}$ models oscillates between 1.6 and 1.9, while the speedup in $\Box_{100\mathrm{M}}$ models rises steadily.

E. Counting Model Parameters

For all models and their variants, we report the number of trainable, non-embedding parameters, i.e., we exclude the parameters in the input (embedding) and output (unembedding) layers. This convention is proposed by Kaplan et al. (2020), who note that using just non-embedding parameters gives their scaling laws a clearer form. The relatively low importance of the number of the embedding parameters for the final performance has been noted by Lan et al. (2020).

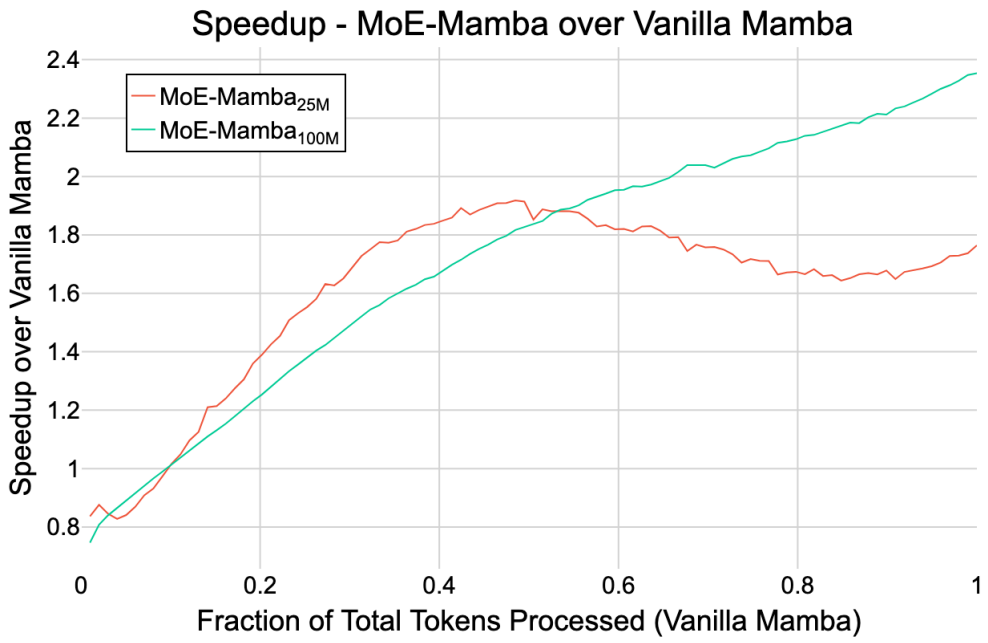

Figure 8. Speedup of different sizes of MoE-Mamba compared to their vanilla Mamba counterparts as training progresses.

F. Exploring the Optimal Mamba to MoE Active Parameters Ratio

The assignment of FLOPs and parameters to different components is an important design choice in heterogeneous architectures. For example, in Transformer, the shape of the model has been studied extensively by Kaplan et al. (2020). In our work, we investigate the optimal ratio of active parameters in the Mamba layer to the number of active parameters in the MoE layer, see section 4.3. Figure 5 may suggest that increasing the ratio strengthens the performance and maybe assigning all the active parameters to Mamba would result in the best performance (ratio *"6:0"*). It should, however, be noted, that all the investigated models contain the same number of both total parameters and active parameters per token. A hypothetical model described above could not achieve this property. If we loosen the requirement and place all the parameters in Mamba, the resulting model is the same as $\text{Mamba}_{25\text{M}}$ with the expansion factor $E = 4$ and 8 instead of 16 Mamba layers. This model achieves marginally worse final log perplexity than $\text{Mamba}_{25\text{M}}$ (3.73).

G. Train and Test Set Performance

In the main text, we report the loss values obtained on the train set. Our training procedure samples from the dataset, so even without processing more tokens than there are in the C4 dataset, the same documents may be encountered multiple times. However, as we process less than 20% of the tokens, the difference in performance on the train set and on the test set should be negligible. For transparency, we provide the results on the test set as well (Figure 9). Their variance may be high due to a limited number of sequences in each evaluation step.

H. Contributions

Maciej integrated Mamba into the codebase, ran experiments related to various aspects of this work, and oversaw the course of the project. Kamil ran the bulk of the experiments. Krystian explored alternative Mamba block designs with Jan's help. Michał and Jakub contributed to the project in various ways, mostly by running experiments and perfecting the codebase. Szymon contributed to our shared repo and participated in some early discussions on integrating MoE and SSMs. Piotr and Marek provided high-level scientific advice. Sebastian supervised the project, setting the research direction and leading experiments and analyses.

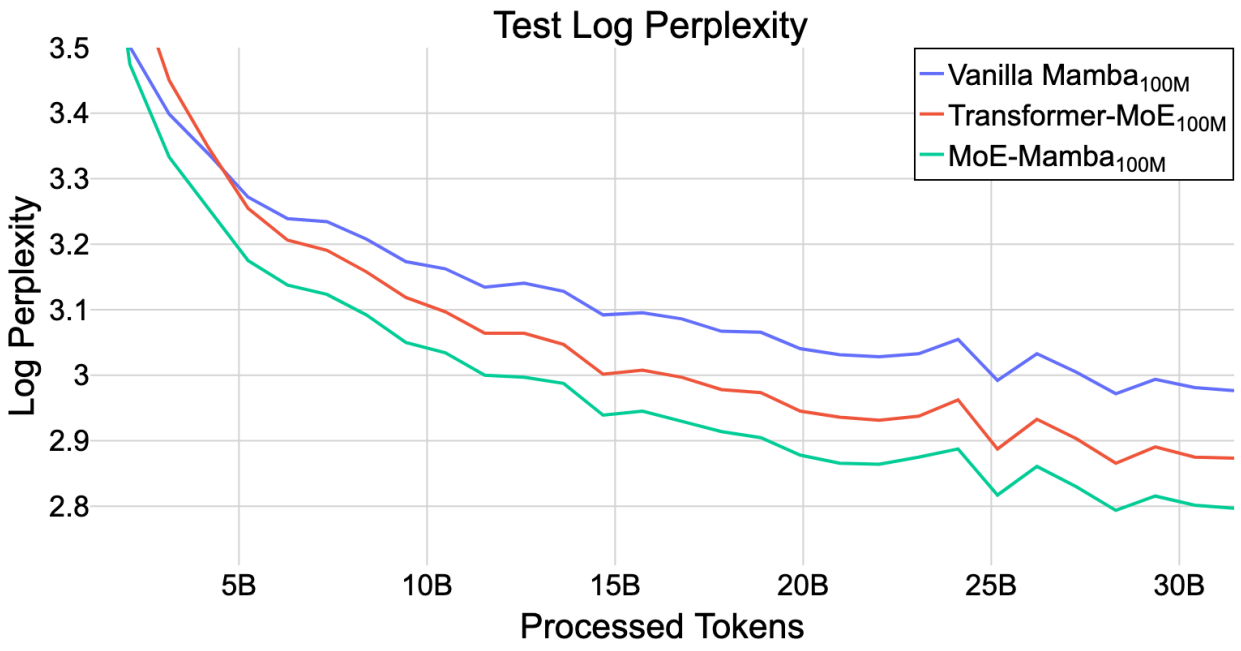

Figure 9. Test set loss.

I. Reproducibility

The codebase used to run the experiments is available at https://github.com/llm-random/llm-random.